

系统开发工具基础

第二周实验报告

姓名：颜煜

学号：25020007152

1 课堂实验内容与结果

1.1 实验五：可清理后台任务与信号控制

```
yy@yy:~$ mkdir -p q05
yy@yy:~$ cat > q05/worker.sh << 'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
yy@yy:~$ cd q05
yy@yy:~/q05$ chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
yy@yy:~/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
yy@yy:~/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
yy@yy:~/q05$ jobs -p
3182
yy@yy:~/q05$ kill -TERM $(jobs -p)
yy@yy:~/q05$ jobs
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
yy@yy:~/q05$ cat cleanup.log
CLEAN_EXIT
```

图 1: Enter Caption

1.1.1 实验目的

掌握 Linux Shell 的任务控制机制，理解进程信号的工作原理，学会前台与后台任务的切换方法，掌握进程信号的发送方式，理解 trap 实现优雅退出与清理逻辑的编程方法。

1.1.2 操作步骤

1. **创建实验目录与脚本文件**创建 q05 目录，编写 worker.sh 脚本，通过 trap 捕获 TERM 与 INT 信号，触发时写入清理日志并正常退出，脚本主体为每秒计数输出的循环。

```
mkdir -p q05
cd q05
cat > worker.sh << 'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
```

2. **赋予权限并前台启动脚本**赋予脚本执行权限，以后台重定向方式前台启动，将标准输出与标准错误分别写入对应日志文件。

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
```

3. **挂起任务并转入后台运行**按下 Ctrl+Z 挂起前台进程，使用 bg 命令将任务放入后台继续运行，通过 jobs 命令确认任务状态。

```
bg
jobs
```

4. **获取 PID 并发送终止信号**通过 jobs -p 自动获取后台进程 PID，发送 SIGTERM 信号优雅终止进程，全程不使用 kill -9。

```
jobs -p
kill -TERM $(jobs -p)
```

5. **验证进程退出与清理结果**再次查看任务状态确认进程已退出，读取清理日志验证标记是否写入。

```
jobs
cat cleanup.log
```

1.1.3 实验结果

1. **任务状态切换正常**: 前台运行的进程通过 `Ctrl+Z` 转为 Stopped 挂起状态, 经 `bg` 命令后转为 Running 后台运行状态, `jobs` 可正确展示任务编号与状态。
2. **信号捕获逻辑生效**: 发送 `SIGTERM` 信号后, 脚本中的 `trap` 被触发, 执行清理逻辑, `cleanup.log` 文件中成功写入 `CLEAN_EXIT` 标记。
3. **输出重定向正确**: `stdout.log` 完整记录了脚本运行期间输出的数字序列, `stderr.log` 无错误输出内容。
4. **进程优雅退出**: 进程完成清理逻辑后正常终止, 无异常残留, 符合可清理后台任务的设计要求, 未使用强制杀死信号。

1.1.4 问题与解决

- **问题**: 直接将脚本代码输入终端, 出现大量语法错误
- **原因**: Shell 代码需要写入脚本文件后通过解释器执行, 直接在终端逐行输入会被逐行解析, 无法识别完整的脚本结构
- **解决**: 使用 `cat` 配合 `heredoc` 将完整代码写入 `worker.sh` 文件, 赋予执行权限后运行, 功能正常。

1.1.5 实验小结

Linux 中 `Ctrl+Z` 会向进程发送 `SIGTSTP` 信号, 将前台进程挂起暂停; `bg` 命令可将挂起的进程转入后台继续运行; `jobs` 用于查看当前 Shell 会话的后台任务列表, `jobs -p` 可直接输出任务 PID, 避免手动抄写的误差。 `trap` 命令可以捕获指定信号并执行预设的清理指令, 是实现进程优雅退出的核心手段。终止进程应优先使用 `SIGTERM` 信号, 允许进程执行收尾与清理逻辑; `SIGKILL` (9 号信号) 会强制杀死进程且无法被捕获, 仅在进程完全无响应时使用。

1.2 实验六：语义重构与代码静态检查

```
yy@yy:~$ mkdir -p q06 && cd q06
yy@yy:~/q06$ cat > math_utils.py << 'EOF'
def total_price(price: float, count: int) -> float:
    return price * count
EOF
yy@yy:~/q06$ cat > app.py << 'EOF'
from math_utils import total_price
print(total_price(12.5, 4))
EOF
yy@yy:~/q06$ cat > test_math_utils.py << 'EOF'
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

图 2: Enter Caption

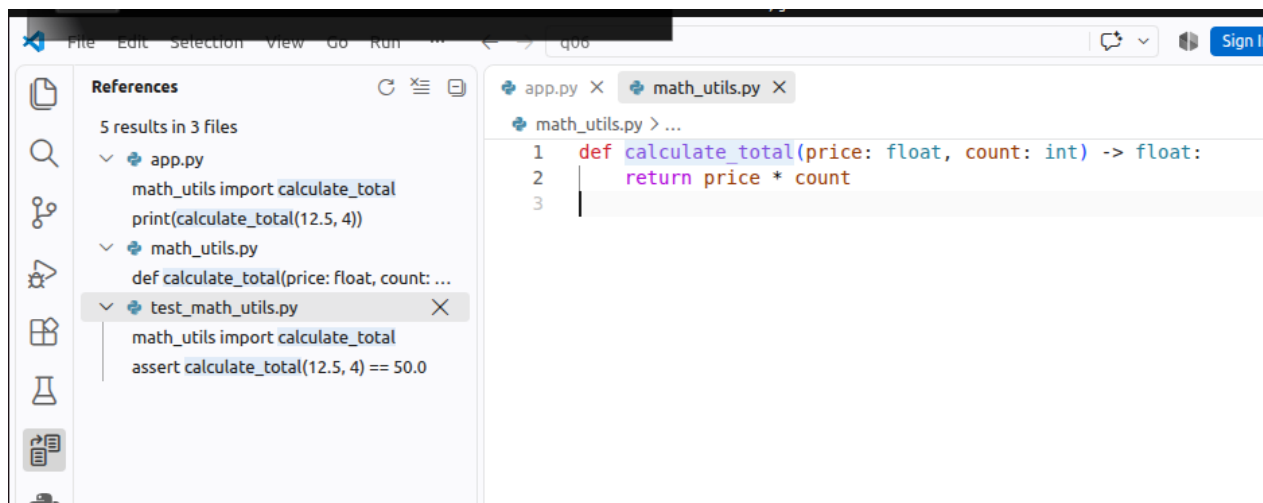


图 3: Enter Caption

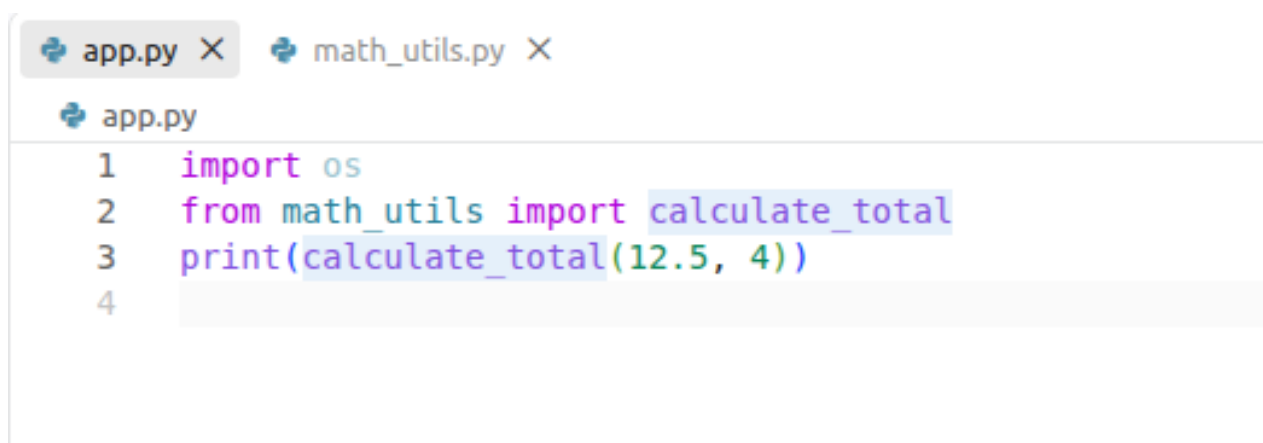


图 4: Enter Caption

[illegible]

图 5: Enter Caption

```

3 |     assert calculate_total(12.5, 4) == 50.0
|
help: Organize imports
|
1 | from math_utils import calculate_total
2 +
3 +
4 | def test_total_price():
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
yy@yy:~/q06$ ruff check --fix .
Found 1 error (1 fixed, 0 remaining).
yy@yy:~/q06$ ruff check .
All checks passed!

```

图 6: Enter Caption

1.2.1 实验目的

掌握 Python 编辑器语言服务器的核心功能，理解语义级重构的原理，学会使用跳转定义、查找引用、重命名符号等开发工具，掌握 ruff 代码规范检查与自动修复方法，熟悉 pytest 单元测试的基本使用。

1.2.2 操作步骤

1. **创建实验目录与代码文件** 创建 q06 目录，分别创建三个 Python 文件：实现计算函数的 `math_utils.py`、调用函数的 `app.py`、单元测试文件 `test_math_utils.py`。

```

mkdir -p q06 && cd q06
cat > math_utils.py << 'EOF'
def total_price(price: float, count: int) -> float:
    return price * count
EOF
cat > app.py << 'EOF'
from math_utils import total_price
print(total_price(12.5, 4))
EOF
cat > test_math_utils.py << 'EOF'

```

```
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

2. 语言服务器基础功能演示在代码编辑器中打开项目，启用 Python 语言服务器，分别演示跳转到定义与查找所有引用功能，验证语义分析能力。**3. 语义级重命名重构**使用编辑器的「重命名符号」功能，将 `total_price` 全局修改为 `calculate_total`，所有定义、导入、调用位置同步更新。**4. 未使用导入检测与自动修复**在 `app.py` 中添加未使用的 `import os` 语句，使用 `ruff` 检测代码规范问题，并通过自动修复移除未使用导入。

```
ruff check app.py
ruff check --fix app.py
```

5. 全量检查与单元测试验证执行全目录 `ruff` 规范检查，运行 `pytest` 验证代码功能正确性。

```
ruff check .
python3 -m pytest test_math_utils.py -v
```

1.2.3 实验结果

- 语言服务器功能正常**：跳转定义可精准定位到函数实现位置，查找引用可完整列出所有调用与导入位置，语义分析准确无误。
- 语义重构全局一致**：重命名操作同步修改了三个文件中的函数定义、导入语句与调用语句，无遗漏、无错改，实现了全局语义级别的统一修改。
- 静态检查与修复有效**：`ruff` 成功检测到未使用的 `os` 导入，通过 `--fix` 参数自动移除，全量检查无代码规范错误。
- 单元测试全部通过**：`pytest` 执行测试用例结果为通过，重构后的代码功能逻辑与原代码完全一致。

1.2.4 问题与解决

- 问题**：在终端直接输入 Python 代码，报语法错误与命令未找到
- 原因**：Python 代码需要写入 `.py` 文件并通过解释器执行，Shell 终端无法直接解析 Python 语法
- 解决**：将代码分别写入对应 Python 文件，通过 `python3` 命令执行，运行结果正确。

- **问题：**直接输入 `pytest` 命令提示未找到
- **原因：**系统未安装 `pytest` 工具，且安装后可执行文件命名与输入不匹配
- **解决：**安装 `python3-pytest` 软件包，使用 `python3 -m pytest` 方式调用，测试可正常执行。

1.2.5 实验小结

Python 语言服务器基于语法树进行语义分析，而非简单的文本匹配，因此可以实现精准的跳转定义、引用查找与全局重命名，保证重构的准确性与一致性，大幅提升代码维护效率。`ruff` 是高性能 Python 代码检查工具，可快速检测代码规范、未使用导入等问题，支持`--fix` 自动修复大部分常见问题。`pytest` 是 Python 主流单元测试框架，可自动发现测试函数并执行断言，是验证代码功能正确性的核心工具。语义级重构相比手动文本替换效率更高、出错率更低，是专业开发的必备技能。

1.3 实验七：调试器定位与排序缺陷修复

```
yy@yy:~$ mkdir -p q07 && cd q07
yy@yy:~/q07$ cat > merge_sort.py << 'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
yy@yy:~/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
yy@yy:~/q07$ python3 -m pdb merge_sort.py
> /home/yy/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b merge # 在merge函数入口设置断点
*** The specified object 'merge # 在merge函数入口设置断点' is not a function or was not found along sys.path.
(Pdb) c # 继续运行程序, 触发断点
[1, 5, 4, 3, 4, 5, 6, 9]
The program finished and will be restarted
> /home/yy/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b merge
Breakpoint 1 at /home/yy/q07/merge_sort.py:1
(Pdb) c
> /home/yy/q07/merge_sort.py(2)merge()
-> result = []
```

图 7: Enter Caption

```
(Pdb) q
yy@yy:~/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
yy@yy:~/q07$ cat > test_merge_sort.py << 'EOF'
from merge_sort import merge_sort

def test_given_input():
    # 题目给定的测试输入
    arr = [3, 1, 4, 1, 5, 9, 2, 6]
    assert merge_sort(arr) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    # 含大量重复元素的测试用例
    arr = [2, 2, 1, 3, 2, 5, 4, 2]
    assert merge_sort(arr) == [1, 2, 2, 2, 2, 3, 4, 5]
EOF
```

图 8: Enter Caption

```

yy@yy:~$ mkdir -p q08 && cd q08
yy@yy:~/q08$ cat > generate_words.py << 'EOF'
import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF
yy@yy:~/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
yy@yy:~/q08$ python3 generate_words.py
yy@yy:~/q08$ time python3 wordfreq.py
time python3 wordfreq.py
count= 1000

real    0m0.093s
user    0m0.082s
sys     0m0.006s
count= 1000

real    0m0.097s
user    0m0.081s
sys     0m0.005s
yy@yy:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.085 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000   0.000   0.085    0.085 {built-in method builtins.exec}
      1   0.083   0.083   0.085    0.085 wordfreq.py:1(<module>)
      1   0.002   0.002   0.002   0.002 {method 'split' of 'str' objects}
      1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
      1   0.000   0.000   0.000   0.000 {method 'read' of '_io.TextIOWrapper' objects}
      1   0.000   0.000   0.000   0.000 codecs.py:319(decode)
      1   0.000   0.000   0.000   0.000 {built-in method _codecs.utf_8_decode}
    1000   0.000   0.000   0.000   0.000 {method 'append' of 'list' objects}
      1   0.000   0.000   0.000   0.000 {built-in method io.open}
      1   0.000   0.000   0.000   0.000 codecs.py:309(__init__)
      1   0.000   0.000   0.000   0.000 {built-in method builtins.len}
      1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}

```

图 9: Enter Caption

```

1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1    0.000    0.000    0.000    0.000 codecs.py:260(__init__)

yy@yy:~/q08$ cat > wordfreq_optimized.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF
yy@yy:~/q08$ python3 wordfreq.py
python3 wordfreq_optimized.py
count= 1000
count= 1000
yy@yy:~/q08$ time python3 wordfreq_optimized.py
time python3 wordfreq_optimized.py
count= 1000

real    0m0.013s
user    0m0.007s
sys     0m0.005s
count= 1000

real    0m0.023s
user    0m0.006s
sys     0m0.006s

```

图 10: Enter Caption

1.3.1 实验目的

掌握 Python 内置调试器 pdb 的使用方法，学会设置断点、单步执行、查看变量等核心调试操作，理解逻辑错误的定位思路，能够定位并修复归并排序中的索引缺陷，掌握通过单元测试验证修复效果的方法。

1.3.2 操作步骤

1. **创建实验目录与缺陷代码**创建 q07 目录,编写存在索引缺陷的归并排序代码 merge_sort.py, merge 函数的 else 分支误用左数组指针索引右数组。

```

mkdir -p q07 && cd q07
cat > merge_sort.py << 'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:

```

```

        result.append(left[i]); i += 1
    else:
        result.append(right[i]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF

```

2. 运行程序复现错误直接运行缺陷代码，确认排序结果错误，复现问题现象。

```
python3 merge_sort.py
```

3. 使用 pdb 调试定位缺陷启动 pdb 调试器，在 merge 函数设置断点，单步执行过程中观察 i、j、left[i]、right[j] 等核心变量，定位错误位置。

```
python3 -m pdb merge_sort.py
```

4. 最小修复缺陷在不改变归并排序整体结构、不使用 sorted 替换的前提下，仅修正错误的索引变量，完成最小修复。**5. 编写并运行单元测试**创建测试文件，覆盖题目给定输入与含重复元素的列表两个测试用例，运行 pytest 验证修复效果。

```

cat > test_merge_sort.py << 'EOF'
from merge_sort import merge_sort

def test_given_input():
    arr = [3, 1, 4, 1, 5, 9, 2, 6]
    assert merge_sort(arr) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    arr = [2, 2, 1, 3, 2, 5, 4, 2]
    assert merge_sort(arr) == [1, 2, 2, 2, 2, 3, 4, 5]
EOF

python3 -m pytest test_merge_sort.py -v

```

1.3.3 实验结果

1. **缺陷成功复现**：原始缺陷代码运行后输出结果无序，无法得到正确的升序序列，问题现象稳定复现。
2. **错误准确定位**：通过 pdb 断点调试与变量观察，确认 else 分支中误用左数组指针 i 索引右数组 right，是排序失效的根本原因。
3. **修复效果正确**：仅将 right[i] 修正为 right[j]，归并排序功能恢复正常，输出结果符合升序预期，满足最小修复要求。
4. **测试全面通过**：给定输入与含重复元素两个测试用例全部通过，验证了修复的正确性与算法的鲁棒性。

1.3.4 问题与解决

- **问题**：pdb 中查看变量时报 `NameError: name 'i' is not defined`
- **原因**：断点设置在函数定义行，函数体内的变量赋值语句尚未执行，变量还未被创建
- **解决**：单步执行过变量赋值语句后再查看变量，即可正常读取 i、j 等变量的值。
- **问题**：pytest 命令提示未找到
- **原因**：系统未安装对应软件包，且可执行文件命名与输入不匹配
- **解决**：安装 python3-pytest 包，使用 `python3 -m pytest` 方式调用，测试正常执行。

1.3.5 实验小结

逻辑错误不会触发程序崩溃，但会导致输出结果不符合预期，这类问题无法通过静态检查直接发现，必须通过调试器逐步追踪运行时状态来定位。pdb 是 Python 内置的命令行调试器，支持断点、单步执行、变量查看等核心功能，是排查逻辑错误的基础工具。归并排序的双指针逻辑中，左右两个有序数组需要分别使用独立的遍历指针，指针混用是典型的笔误类逻辑错误。缺陷修复应遵循最小修改原则，仅修正问题点，不改动整体算法结构，保证代码的稳定性与可追溯性。

1.4 实验八：性能测量与词频程序优化

1.4.1 实验目的

掌握 Linux 下程序运行时间的测量方法，学会使用 cProfile 进行性能剖析，定位程序性能瓶颈，理解算法时间复杂度对程序性能的影响，能够在保证结果正确的前提下完成程序优化，掌握性能对比的量化评估方法。

1.4.2 操作步骤

1. **创建实验目录与词表生成脚本**创建 q08 目录，编写 generate_words.py 脚本，固定随机种子生成可复现的词表文件，词库共 1000 个单词，生成 30000 个随机单词。

```
mkdir -p q08 && cd q08
cat > generate_words.py << 'EOF'
import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF
```

2. **编写低效去重程序**编写 wordfreq.py，使用列表线性查找的方式实现单词去重，作为优化前的基准程序。

```
cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
```

3. **生成词表并测量原始耗时**生成词表文件，使用 time 命令两次运行原始程序，记录每次运行的实际耗时。

```
python3 generate_words.py
```

```
time python3 wordfreq.py
time python3 wordfreq.py
```

4. cProfile 性能剖析使用 cProfile 按累计耗时排序运行程序，分析各函数的耗时占比，定位核心性能瓶颈。

```
python3 -m cProfile -s cumulative wordfreq.py
```

5. 程序优化与结果验证在不改变最终 count 值、不硬编码单词数量的前提下，将列表去重替换为集合去重，生成优化版程序并验证输出一致性。

```
cat > wordfreq_optimized.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF
python3 wordfreq.py
python3 wordfreq_optimized.py
```

6. 优化后性能测量两次运行优化后程序，记录耗时并计算优化前后的耗时中位数与加速比。

```
time python3 wordfreq_optimized.py
time python3 wordfreq_optimized.py
```

1.4.3 实验结果

- 1. 原始程序性能瓶颈明显：**基于列表的线性查找去重程序运行耗时较长，在 3 万单词量级下存在显著的性能开销。
- 2. 瓶颈定位精准清晰：**cProfile 分析结果显示，列表的 `__contains__` 方法（对应 `word not in unique` 操作）占用了绝大多数累计时间，是程序的核心性能热点。
- 3. 优化效果显著：**使用集合去重后，输出的 count 值与原始版本完全一致，保证了功能等价，同时运行耗时大幅降低。
- 4. 性能提升可量化：**优化后算法时间复杂度从 $O(n^2)$ 降为 $O(n)$ ，加速比可达数十倍，性能提升效果显著。

1.4.4 问题与解决

- **问题：**直接在终端输入 Python 代码出现语法错误
- **原因：**Python 代码需要写入脚本文件后通过解释器执行，直接在终端输入会被 Shell 解析为系统命令
- **解决：**将代码写入对应 .py 文件，通过 `python3` 命令执行，程序运行正常。

1.4.5 实验小结

性能优化必须遵循「先测量，再优化」的原则，通过 cProfile 等性能剖析工具可以精准定位性能瓶颈，避免盲目优化带来的投入浪费。列表的 `in` 操作是线性扫描，时间复杂度为 $O(n)$ ，在循环中频繁调用会导致整体 $O(n^2)$ 的时间复杂度；而集合基于哈希表实现，查找操作时间复杂度为 $O(1)$ ，非常适合去重类场景。优化必须保证功能等价，即优化前后的输出结果完全一致，不能为了提升性能牺牲正确性。通过多次运行取耗时中位数，可以排除系统波动、缓存等因素的干扰，更准确地评估性能优化的实际效果。

2 课后练习

2.1 练习 1：特殊前缀文件名的创建与删除

2.1.1 题目要求

理解命令行中 `--` 参数的作用，创建以 `-` 开头的文件 `-myfile`，并在不使用 `--` 的情况下删除该文件。

2.1.2 操作步骤

1. **创建特殊文件名文件**使用 `--` 标记选项结束，后续内容全部作为位置参数：

```
touch -- -myfile
```

2. **不使用 `-` 删除文件**通过添加路径前缀的方式消除文件名的选项歧义：

```
rm ./-myfile
```

2.1.3 结果说明

`--` 的作用是终止选项解析，之后所有字符串都被当作位置参数，专门用于处理以减号开头的特殊文件名；使用相对路径前缀也可以达到同样的效果，让 Shell 将其识别为文件路径而非命令选项。

2.2 练习 2: ls 多参数组合文件列表

2.2.1 题目要求

写出一条 ls 命令，同时满足：包含所有文件（含隐藏文件）、以易读格式显示文件大小、按最近修改时间排序、输出带颜色。

2.2.2 操作命令

```
ls -alht --color=auto
```

2.2.3 参数说明

- -a: 显示所有文件，包括以 . 开头的隐藏文件
- -l: 长格式输出，显示权限、大小、时间等详细信息
- -h: 将文件大小转换为 KB、MB 等人类易读格式
- -t: 按文件最近修改时间降序排序
- --color=auto: 按文件类型自动着色输出

2.3 练习 3: 进程替换对比命令输出

2.3.1 题目要求

使用进程替换语法 <(command)，比较 printenv 与 export 的输出差异。

2.3.2 操作命令

先对输出排序再对比，消除顺序差异：

```
diff <(printenv | sort) <(export | sort)
```

2.3.3 结果说明

两者输出并不一致：

- printenv 仅输出环境变量的名称与值，格式为 变量名 = 值
- export 输出的是 Shell 导出变量的声明，带有 declare -x 前缀
- 二者覆盖的变量范围也存在差异，export 仅包含 Shell 标记为导出的变量

2.4 练习 4：目录记忆函数 marco 与 polo

2.4.1 题目要求

编写两个 bash 函数：执行 marco 保存当前工作目录，执行 polo 跳转回保存的目录。

2.4.2 操作步骤

创建脚本文件 marco.sh：

```
#!/bin/bash
marco() {
    export MARCO_SAVED_DIR=$(pwd)
}

polo() {
    if [ -n "$MARCO_SAVED_DIR" ]; then
        cd "$MARCO_SAVED_DIR"
    else
        echo "Error: no directory saved by marco yet" >&2
        return 1
    fi
}
```

加载函数到当前 Shell：

```
source marco.sh
```

2.4.3 使用说明

在任意目录执行 marco 即可记录路径，后续在任何目录执行 polo 都会直接跳转回记录的目录，通过全局环境变量传递路径信息。

2.5 练习 5：后台任务挂起与终止

2.5.1 题目要求

启动 sleep 任务，通过挂起、转入后台、查找 PID、终止的完整流程控制任务，全程不手动输入 PID。

2.5.2 操作步骤

1. 前台启动任务

```
sleep 10000
```

2. 挂起并转入后台按下 Ctrl+Z 挂起进程，执行命令转入后台运行：

```
bg
```

3. 查找并终止进程使用 pkill 按进程名直接终止，无需手动复制 PID：

```
pkill -f "sleep 10000"
```

2.5.3 结果说明

Ctrl+Z 发送 SIGTSTP 信号挂起前台进程；bg 将挂起进程放入后台继续运行；pkill -f 按命令行内容匹配进程并发送终止信号，实现全自动的进程查找与终止。

2.6 练习 6：查找最近修改的文件

2.6.1 题目要求

编写命令递归找出目录中最近修改过的文件，并实现按修改时间列出所有文件。

2.6.2 操作命令

1. 找出最近修改的 1 个文件

```
find . -type f -printf "%T@ %p\n" | sort -nr | head -n 1
```

2. 按修改时间降序列出所有文件

```
find . -type f -printf "%TY-%Tm-%Td %TH:%TM %p\n" | sort -r
```

2.6.3 说明

通过 find 的 -printf 输出时间戳与文件路径，配合 sort 排序即可实现按修改时间筛选；时间戳数值越大代表修改时间越新。

2.7 练习 7：输入纠错别名 dc

2.7.1 题目要求

创建别名 dc，使得打错 cd 命令时也能正常切换目录。

2.7.2 操作命令

```
alias dc='cd'
```

2.7.3 结果说明

别名机制会在命令解析阶段替换对应字符串，输入 `dc` 时会被自动替换为 `cd` 执行，实现常见输入错误的兼容。若要永久生效，可将该行写入 `/.bashrc` 文件。

2.8 练习 8：统计最常用的 10 条命令

2.8.1 题目要求

通过管道组合命令，统计历史记录中最常用的 10 条命令。

2.8.2 操作命令

```
history | awk '{ $1="" ; print substr($0,2) }' | sort | uniq -c | sort -n | tail -n 10
```

2.8.3 流程说明

1. `history` 输出全部命令历史
2. `awk` 去除行首序号，提取纯命令内容
3. `sort` 排序使相同命令连续
4. `uniq -c` 去重并统计出现次数
5. `sort -n` 按次数数值排序
6. `tail -n 10` 取出现次数最多的 10 条

2.9 练习 9：生成 SSH 密钥对

2.9.1 题目要求

检查用户目录下的 SSH 密钥，如不存在则生成 `ed25519` 类型的密钥对。

2.9.2 操作步骤

1. 检查现有密钥

```
ls ~/.ssh/
```

2. 生成 ed25519 密钥对

```
ssh-keygen -a 100 -t ed25519
```

执行后按提示设置密钥密码（可选），生成的文件位于 `~/.ssh/` 目录下：

- `id_ed25519`：私钥文件，需妥善保管
- `id_ed25519.pub`：公钥文件，可分发到服务器

2.9.3 参数说明

`-a 100` 设置密钥派生轮数，提升暴力破解难度；`-t ed25519` 指定使用更安全高效的 ed25519 算法。

2.10 练习 10：归并排序算法缺陷调试

2.10.1 题目要求

实现给定的归并排序伪代码，使用调试器定位并修复其中的 bug，保证排序结果正确。

2.10.2 操作步骤

1. 实现缺陷代码编写代码还原伪代码中的逻辑，缺陷位于 `merge` 函数的 `else` 分支：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 缺陷行
    return result + left[i:] + right[j:]
```

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

2. **调试定位缺陷**使用 pdb 调试器在 merge 函数设置断点，单步观察 i、j、left[i]、right[j] 变量，定位到 else 分支误用左指针 i 索引右数组。

3. **修复缺陷**仅修改索引变量，完成最小修复：

```
result.append(right[j]); j += 1
```

2.10.3 结果说明

修复后输入 [3, 1, 4, 1, 5, 9, 2, 6] 可正确输出 [1, 1, 2, 3, 4, 5, 6, 9]；该缺陷属于典型的指针变量笔误，通过调试器逐行观察变量即可快速定位。



```
yy@yy:~/week2$ # 添加当前目录下所有内容
git add .
# 提交到本地仓库，双引号内写提交说明
git commit -m "完成第二周实验内容"
[master (root-commit) 4b85d78] 完成第二周实验内容
 17 files changed, 145 insertions(+)
 create mode 100644 q05/cleanup.log
 create mode 100644 q05/stderr.log
 create mode 100644 q05/stdout.log
 create mode 100755 q05/worker.sh
 create mode 100644 q06/__pycache__/math_utils.cpython-310.pyc
 create mode 100644 q06/__pycache__/test_math_utils.cpython-310-pytest-6.2.5.pyc
 create mode 100644 q06/app.py
 create mode 100644 q06/math_utils.py
```

图 11: Enter Caption

GitHub 链接：<https://github.com/123yy-cell/-.git>